



Facultad de Ingeniería
Lógica

Proyecto Grafos

Integrantes

Valentina Penen
Virginia Cota
Blas Melissari

Semestre 1 de 2026

Introducción

El objetivo del proyecto fue implementar distintas operaciones sobre diversos grafos usando Prolog, integrando lógica proposicional, lógica de predicados y programación lógica.

Para resolver el problema planteado, se decidió generar un sistema del estilo GPS. Los Nodos los consideramos como cuadras y los vértices como calles. Cada calle tiene un origen, un destino y un peso.

La idea general fue separar el proyecto en varios archivos, donde cada uno se encarga de una parte distinta: carga de datos, búsqueda de rutas, conectividad, ruta más corta, diámetro, flujo máximo e interfaz.

Representación del grafo y carga de datos

La carga de grafos se define en el archivo datos.pl. Allí se usan dos predicados dinámicos: manzana/3 y calle/4. Estos predicados son los que guardan la información del grafo dentro de la base de conocimiento de Prolog.

El predicado manzana/3 guarda el identificador de la manzana y sus coordenadas [manzana(Id, X, Y)]. El predicado calle/4 guarda el nombre de la calle, el nodo de origen, el nodo de destino y el peso [calle(Nombre, Origen, Destino, Peso)]. Ese peso se usa como costo para calcular caminos y, en el caso del flujo máximo, se interpreta como capacidad.

Para cargar los datos se usan archivos CSV. Dentro de nuestro sistema los grafos disponibles son :

- Un grafo 26x26 Dirigido, cíclico y no convexo.
- Un grafo 6x6 Dirigido y cíclico.
- Un grafo 4x4 No Dirigido.

Organización de archivos

El proyecto está dividido en archivos separados, cada uno con una función concreta.

Archivo	Función dentro del proyecto
main.pl	Carga los módulos y define los predicados inicializar/0 e inicializar_interfaz/0

datos.pl	Carga y guarda manzanas y calles como hechos dinámicos
rutas_posibles.pl	Busca rutas posibles entre dos nodos usando DFS
conectividad.pl	Verifica si existe un camino entre dos nodos
ruta_mas_corta.pl	Calcula el peso total de una ruta y busca la más eficiente por peso.
Dijkstra.pl	Implementa el algoritmo de Dijkstra para la ruta de menor costo
ruta_mas_short.pl	Calcula distancias según cantidad de aristas
diametro.pl	Calcula el diámetro del grafo
ford_fulkerson.pl	Implementa el cálculo de flujo máximo para calcular la ruta más corta
interfaz.pl	Crea la interfaz gráfica con XPCE

Rutas posibles

Para encontrar todas las rutas entre dos manzanas se usa una búsqueda en profundidad, también llamada DFS. Este algoritmo avanza lo más profundo posible en cada rama y luego retrocede (Backtracking) , y prueba con la siguiente.

Para esto utilizamos recursión, el caso base se da cuando el nodo actual ya es el destino. En ese momento la ruta queda encontrada. En el caso recursivo, se toma una calle desde el nodo actual hacia un nodo siguiente y se continúa buscando desde ahí.

Para que el algoritmo no quede dando vueltas en ciclos, se usa una lista de visitados. Antes de avanzar a un nodo siguiente, se verifica que ese nodo todavía no haya sido visitado.

El cálculo de todas las rutas posibles solo está disponible para los grafos de 6x6 y 4x4 dado que el crecimiento de la cantidad de caminos es exponencial comparado al crecimiento del grafo. Aumentar la cantidad de nodos genera que la computadora tarde/ no pueda calcularlo.

Conectividad

La conectividad se resuelve en el archivo `conectividad.pl` con el predicado `conectado/2`. Este predicado reutiliza `dijkstra/4` para verificar si existe una ruta entre el origen y el destino. Si Dijkstra encuentra algún camino, entonces los nodos se consideran conectados.

Diametro

Como para este algoritmo el peso es irrelevante, hicimos un nuevo algoritmo que llamamos `ruta_mas_short/4`. Lo que hace este es calcular todos los caminos posibles entre dos nodos y fijarse la cantidad de aristas de cada uno.

Luego tenemos el `diametro/3` que lo que hace es comparar los caminos más cortos entre todos los pares de nodos, implementando `ruta_mas_short/4`. De estos elige el más largo y este es el diámetro del grafo.

Rutas con peso y Dijkstra

Además de encontrar rutas, el programa también puede calcular cuánto pesa una ruta, para encontrar la más eficiente. Tenemos dos formas de calcularlo.

La forma menos eficiente de encontrar la ruta más eficiente es calcular todas las rutas, calcular el peso de todas y agarrar la que tenga el peso menor. Pero por esto, este algoritmo solo lo pudimos implementar en los grafos más chicos.

La forma más eficiente es Dijkstra. Este algoritmo mantiene una frontera de caminos candidatos. Cada candidato guarda el peso acumulado, el nodo actual y la ruta recorrida hasta ese momento. En cada paso se buscan los vecinos del nodo actual, se generan nuevos caminos y se agregan a la frontera. Luego se ordena la frontera por lo que el próximo camino a analizar es el de menor peso acumulado. En la interfaz, el botón de ruta más corta usa `dijkstra/4`.

Interfaz

Nuestra interfaz fue hecha en colaboración con Chat GPT pues era bastante complicada y sentimos que excede los límites del curso y no era el propósito del ejercicio.

Para correr el programa

?- [[main.pl](#)].

?- inicializar_interfaz.

Bibliografía

DFS: https://uchile.progcomp.cl/apunte/grafos/busqueda_en_grafos/

Dijkstra:

<https://www.geeksforgeeks.org/dsa/dijkstras-shortest-path-algorithm-greedy-algo-7/>

Diámetro y grafos en general: https://es.wikipedia.org/wiki/Teor%C3%ADa_de_grafos